

#	Critério de modularização	Gemini	ChatGPT (v2)
1	Decomposição em funções, cada uma com responsabilidade única	2	2
2	Nomes descritivos de funções e variáveis	2	2
3	Ausência de duplicação de código (DRY)	2	2
4	Parâmetros e retornos bem definidos; evita variáveis globais	2	1
5	Reaproveitamento de funções (ex.: validação usada em um só lugar)	2	2
6	Existência de uma função principal organizando o fluxo	2	2
7	Legibilidade geral e comentários úteis (sem excesso)	2	2
Total (máx. 14)		14	13

Conclusão final

1. Os dois são muito bons. Os dois funcionam, são organizados e fáceis de entender.
2. O Gemini ainda ganha por pouco. Ele é um pouquinho melhor porque não usa nenhuma variável global. Tudo entra por parâmetro e sai por retorno. Isso é tecnicamente mais correto.
3. O ChatGPT melhorou muito da primeira versão para essa. A versão 2 está quase no mesmo nível do Gemini. O único "defeito" é que ele ainda usa constantes globais no topo (`TENSAO_NOMINAL = 220`). Se ele tivesse passado esses valores como parâmetro, teria nota máxima também.
4. Na prática, para um sistema real de monitoramento, os dois códigos seriam aceitáveis. Mas se eu fosse engenheiro e tivesse que escolher um, escolheria o Gemini por ser ligeiramente mais flexível e testável.

O que eu aprendi com essa comparação?

- O ChatGPT consegue melhorar se você pedir (ou se ele "aprender" com os erros).
- O Gemini já entregou um código muito bom de primeira.
- Pequenos detalhes, como evitar variáveis globais, fazem diferença na qualidade do código.
- Para a faculdade, os dois códigos são ótimos exemplos de modularização.